

SSC CGL Practice Android App

Focused navigation, screens, and
data-flow audit

READ-ONLY TECHNICAL REPORT

Prepared from the current repository state

Audit scope

Bottom navigation, secondary navigation, screen completeness, current data sources, persistence gaps, and the recommended future flow from Room to repositories to ViewModels to Compose UI.

Project: I:/SSC Exam App/SSC-CGL

Report date: 2026-09-22

Change policy: No project files were modified for this audit.

1. Executive Summary

The four bottom-navigation destinations are wired and switch correctly while the app is in its main-tab container. Home shortcuts also route to Subjects, Test, and Notes. The navigation layer is nevertheless a manual state machine rather than a scalable navigation architecture, and the four sections use inconsistent data paths.

Section	Navigation	Implementation status	Primary data source
Home	Linked and switchable	UI mostly complete; progress is fake	Hard-coded values + SharedPreferences
Subjects	Linked; topic practice opens	Practice works; subtopic route is bypassed	Hard-coded Kotlin repositories
Test	All three menu cards linked	Custom/mock partial; PYQ stops at paper cards	Hard-coded test and question data
Notes	All three categories linked	Most complete persistence path	Room through DailyNotesRoomRepository

Main finding

The UI is ahead of the data architecture. Most screens look functional, but only Notes is substantially backed by Room. Subjects and Tests still use small reusable in-memory question pools, and Home does not consume real progress.

Current navigation architecture

MainActivity owns the current tab, current secondary screen, selected subject/topic/subtopic, test settings, and result objects. It renders every destination from a single when-expression and implements back behavior manually.

- No NavHost or rememberNavController usage was found.
- No ViewModel or viewModel() usage was found.
- The bottom bar is visible only inside MAIN_TABS; secondary screens replace the whole tab container.
- Result objects are held with remember rather than persisted or safely restored.

2. Navigation and Screen Switching

Bottom navigation

BottomNavTab defines Home, Subjects, Test, and Notes. Each NavigationBarItem calls onTabSelected(tab), which replaces currentTab. The selected icon, text color, and indicator update from the same state, so selection and visible content stay synchronized.



Route	Observed behavior	Assessment
Home shortcut -> Subjects	Changes currentTab to SUBJECTS	Correct
Home shortcut -> Test	Changes currentTab to TEST	Correct
Home shortcut -> Notes	Changes currentTab to NOTES	Correct
Subject card -> Topics	Saves subject ID and opens TOPICS	Correct
Topic card -> Practice	Saves topic ID and opens PRACTICE	Works, but skips Subtopics
Tests -> Custom	Opens setup, runner, then result	Functional
Tests -> Mock	Opens overview, runner, then result	Functional but simulated
Tests -> PYQ	Opens years and year detail	Stops before an executable paper
Notes -> category	Switches internal category screen	Correct

Back-stack and restoration risks

- The Subjects screen's visible back action goes to Welcome, while system back from a non-Home main tab returns to Home first.
- Transition direction is inferred from AppScreen enum order, not actual navigation history.
- currentScreen survives saved-state recreation, but practiceResult and customTestResult do not.
- Restoring a result destination without its result object causes a redirect away from that screen.
- No deep links, typed route arguments, or independent tab back stacks are supported.

Navigation defect

AppScreen.SUBTOPICS and a complete SubtopicsScreen exist, but no normal topic click enters that destination. The user-visible flow jumps directly from Topics to Practice.

3. Home Section

What is implemented

- Personalized greeting using the name captured on Welcome.
- Working shortcuts to Subjects, Test, and Notes.
- A Today's Progress card, quick-options area, and Memory Bite card.
- Memory Bite remembered/review-later behavior stored in SharedPreferences.

Hard-coded, incomplete, or missing

- Questions Practiced is always 0 and Accuracy is always 0%.
- No attempt, answer, or time-spent data is read from Room.
- Memory Bite content is a six-item hard-coded list.
- The user's name is not permanent profile data.
- Recent attempts, resume test, streaks, bookmarks, weak topics, and study goals are absent.

Required database integration



HomeViewModel should aggregate today's attempts, correct/wrong totals, time spent, recent activity, incomplete attempts, bookmark count, daily streak, and a recommended revision item. The screen should render a single immutable HomeUiState instead of constants.

Home element	Required source
Questions practiced	Count of today's AttemptAnswer records
Accuracy	Correct / attempted aggregate
Streak	Distinct active study dates
Resume	Latest incomplete Attempt
Recommendation	QuestionProgress grouped by weak topic
Memory/revision	Revision queue and due timestamp

4. Subjects and Practice Section

What is implemented

- Four Tier-I subject cards: Reasoning, Maths, English, and GK/GA.
- Subject to topic navigation.
- Practice runner with answer selection, clear response, previous/next, mark for review, palette, timer, and submit.
- Result analysis with correct, wrong, unattempted, accuracy, time, explanations, and filters.

Broken or misleading behavior

- Topic selection goes directly to Practice instead of Subtopics.
- getQuestionsForSubtopic ignores its requested subtopic and delegates to the full topic query.
- Subtopic labels are often assigned by cycling through a list, not by content meaning.
- Most topics reuse the same generic 20 or 25 questions.
- Retakes return the same questions in the same order.
- Practice results are not persisted to Room.

Actual flow	Subjects -> Topics -> Practice -> Result
Intended flow	Subjects -> Topics -> Subtopics -> Practice -> Result

Required database integration



Layer	Responsibility
SubjectRepository	Subjects, topic counts, subtopic counts, ordering
QuestionRepository	Filtered, randomized, validated question queries
AttemptRepository	Attempt lifecycle, answers, timers, completion
SubjectsViewModel	Subject/topic/subtopic list state
PracticeViewModel	Question selection, response state, palette state, save/restore
ResultViewModel	Persisted scoring and review projections

5. Tests Section

The Test menu correctly links to Custom Test, PYQ Test, and Mock Test. Their depth and data completeness differ substantially.

Custom Test

- Functional: subject selection, select all, timed/untimed runner, palette, scoring, subject breakdown, and review.
- Hard-coded: every selected subject contributes exactly 25 questions and 15 minutes.
- Missing despite the menu description: topic selection, question-count selection, difficulty, PYQ filtering, and randomization.
- The runner loads PracticeQuestionRepository directly and saves no attempt.

Mock Test

- Functional: overview, 100 questions, 60-minute timer, +2/-0.50 scoring, runner, result, and review.
- Partial: Hindi can be selected, but the chosen value is ignored and question content remains English.
- Partial: the same generic subject pools power every mock.
- Missing: per-section timing, section locking, official paper variants, persistence, and resume.

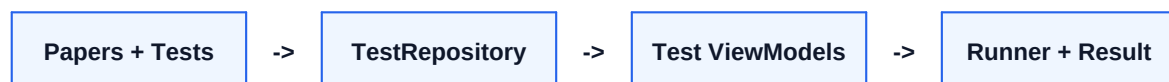
PYQ Test

- Functional: year list, year selection, year details, and shift filters.
- Hard-coded: year, date, shift, duration, and paper metadata.
- Broken endpoint: PyqPaperShiftCard has no start/open callback.
- Missing: actual paper-question relationships and executable PYQ papers.

PYQ endpoint

Tests -> PYQ years -> Year detail -> Shift card -> no action

Required database integration



The existing Room schema already contains tests, test_questions, attempts, and attempt_answers, but none of these tables are connected to the test UI. Future TestRunnerViewModel state should be persisted continuously so timer, answers, and current position survive process loss.

6. Notes Section

What is implemented

- Notes overview with Vocabulary, Idioms, and Current Affairs.
- Internal category navigation and back handling.
- Room-backed active-set loading and completion status.
- Five seeded sets of ten items for each category.
- Completion advances to the next uncompleted set.

Partial or risky behavior

- Compose screens call DailyNotesRoomRepository directly; no NotesViewModel mediates loading and actions.
- The UI marks a whole set complete but does not expose per-item completion or review-later controls.
- Room supports item progress and review-later flags, but those APIs are unused by the UI.
- A legacy DailyNotesRepository still exists, creating an additional source of truth.
- Current Affairs are static seeded entries, not a dated/versioned content feed.
- Repeated seed checks can race; initialization needs one atomic transaction.

Required database integration



NotesViewModel should expose active set, items, completion counts, review-later state, and explicit loading/error states. Current Affairs should use dedicated date, category, source, validity, and verification fields rather than generic note fields.

7. Current and Future Data Flow

Current state

Section	Current path	Persistence
Home	UI -> hard-coded zero metrics / MemoryBiteRepository	SharedPreferences for bites only
Subjects	UI -> SscDataRepository -> PracticeQuestionRepository	None for attempts
Tests	UI -> PracticeQuestionRepository / PyqDataRepository	None for attempts
Notes	UI -> DailyNotesRoomRepository -> AppDatabase	Room set completion

Recommended common architecture



User actions should travel in the reverse direction: UI event -> ViewModel method -> repository transaction -> Room -> observed UI state. Compose should not know about DAOs, database instances, or seeding.

Database area	Consumers
Subjects / topics / subtopics	Subjects and custom-test filters
Questions / options / assets	Practice, custom tests, mocks, PYQ review
Exam papers / sections / mappings	PYQ and official mock simulations
Attempts / answers	Home progress, runner recovery, results, history
Question progress / bookmarks	Home recommendations and Subjects mastery
Notes / current affairs / progress	Notes overview and revision queue

8. Integration Boundaries and Priorities

Priority	Required outcome	Affected sections
P0	One authoritative question/taxonomy source and consistent IDs	Subjects, Test, Home
P0	Persist attempts and answers; expose real Home metrics	Home, Subjects, Test
P1	Restore Topics -> Subtopics -> Practice	Subjects
P1	Connect PYQ paper cards to real stored papers	Test
P1	Add ViewModels between repositories and Compose	All four sections
P1	Make Notes seeding atomic and expose item progress	Notes
P2	Add mock sectional timing and functional language handling	Test
P2	Add process-safe navigation and state restoration	All sections

Correct today

- All four bottom tabs are linked and visually synchronized.
- Home shortcuts switch tabs correctly.
- Subjects can reach topic practice and result analysis.
- Custom and mock test flows reach a runner and result.
- PYQ year-level navigation works.
- Notes categories navigate correctly and read active sets from Room.

Incomplete today

- Subtopics are unreachable and filtering is ineffective.
- Home progress is not real.
- Subjects and Tests are not connected to the Room question schema.
- PYQ shift cards cannot launch a paper.
- Mock language and current sectional behavior are incomplete.
- Attempts, bookmarks, history, and progress analytics are absent.
- No ViewModel layer separates UI from repositories.

Final assessment

Navigation at the tab level is correct. Feature depth and persistence are uneven: Notes is the closest to a real offline database flow, Subjects and Tests remain in-memory demonstrations, and Home is primarily a static dashboard. The next architecture milestone should connect all four sections to one Room-backed repository layer through lifecycle-aware ViewModels.

Appendix A. Principal Files Inspected

```
app/src/main/java/com/example/MainActivity.kt
app/src/main/java/com/example/ui/screens/EmptyTabScreens.kt
app/src/main/java/com/example/ui/screens/GreetingScreen.kt
app/src/main/java/com/example/ui/screens/TopicsScreen.kt
app/src/main/java/com/example/ui/screens/SubtopicsScreen.kt
app/src/main/java/com/example/ui/screens/PracticeQuestionScreen.kt
app/src/main/java/com/example/ui/screens/CustomTestSetupScreen.kt
app/src/main/java/com/example/ui/screens/CustomTestScreen.kt
app/src/main/java/com/example/ui/screens/MockTestOverviewScreen.kt
app/src/main/java/com/example/ui/screens/PyqScreens.kt
app/src/main/java/com/example/data/PracticeQuestionRepository.kt
app/src/main/java/com/example/data/models/SscData.kt
app/src/main/java/com/example/data/local/AppDatabase.kt
app/src/main/java/com/example/data/local/DailyNotesRoomRepository.kt
app/src/main/java/com/example/data/local/SscLocalRepository.kt
```

Audit method: static source inspection and route/data-source tracing. The report describes the repository state after removal of the obsolete SSC CSV datasets. No Android source, resource, navigation, or database file was changed while producing this report.